

Cumhuriyet Üniversitesi İşletim Sistemleri Dersi

Bölüm 3 *Süreç Yönetim Temelleri* *(Process Management)*

Dr. Halil ARSLAN

Bölümün hedefleri

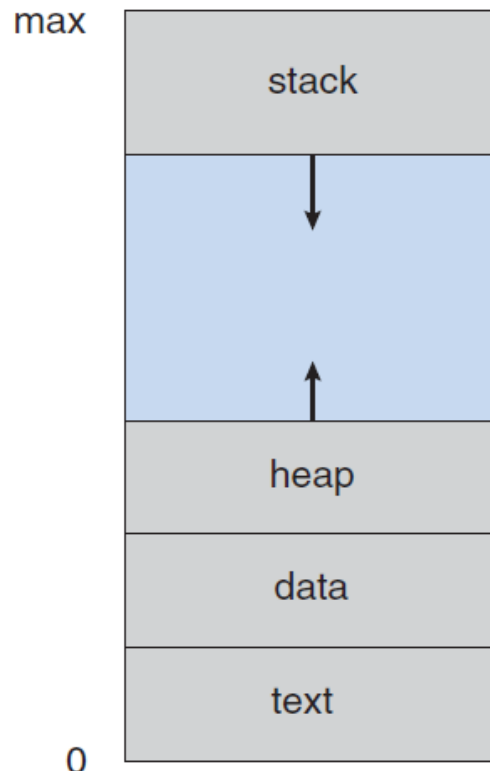
- Süreç/Process kavramı
- Süreç planlama
- Süreç işlemleri
- Süreçler arası iletişim
- Süreç iletişim örnekleri
- İstemci/sunucu iletişim

Süreçler / Processes

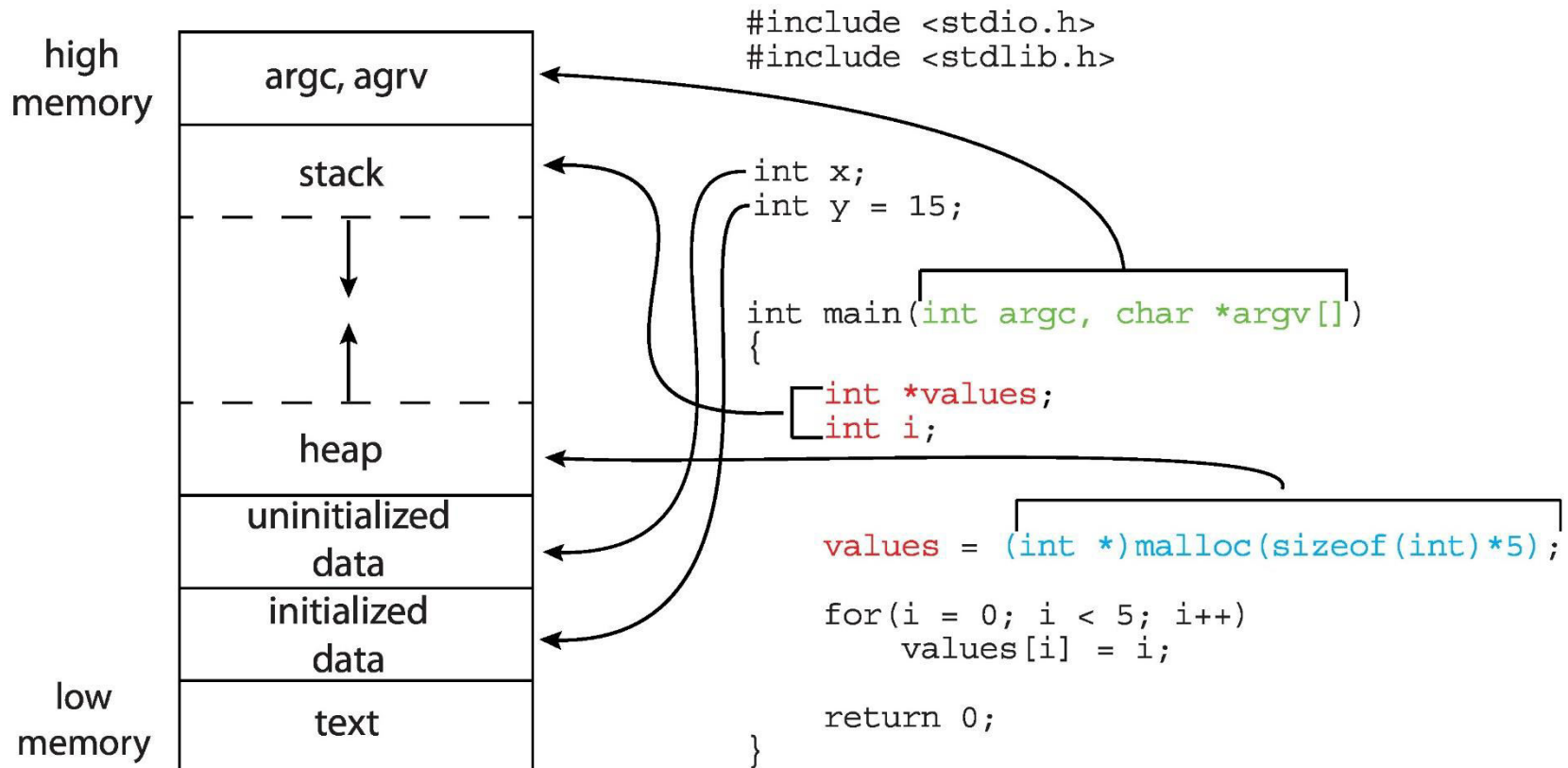
- Bilgisayar sistemleri başlangıçta tek bir süreci yürütebilecek şekilde tasarlanmıştır. Sistemin tüm denetim ve kaynaklara erişim hakkı yürütülen sürece aitti.
- Modern bilgisayar sistemlerinde ise birden fazla program belleğe yüklenerek aynı zamanda yürütülebilmesi sağlanır. Bu yaklaşım süreçler arasında sıkı kontrol ve merkezi süreç yönetimi gereksinimini getirdi.
- Bir program her zaman süreç olarak isimlendirilmez. Bir programın süreç olarak isimlendirilebilmesi için komut setlerinden oluşan derlenmiş komutlar içeren aktif bir varlık olması gerekir.
- Bir program yürütülürken ana belleğe yüklenmesi ile süreç oluşur.
- Bir süreç içerdiği komut setlerinden (text section) daha fazla bir belleğe ihtiyaç duyar.
- Bir süreç genellikle fonksiyon parametreleri, dönüş adresleri, yerel değişkenler ve global değişkenleri içeren **stack** alanına ihtiyaç duyar.
- Bir süreç aynı zamanda çalışma zamanında dinamik olarak ayrılan bir **heap** bellek alanı içerir.

Süreçler / Processes

- **Stack** ve **Heap** alanları, sürece tahsis edilen bellek alanının zıt yönlenrinden başlar. Tahsis edilen alan dolarsa stack dolma (stack overflow) hatası yada malloc hatası oluşacaktır.
- Süreçler bellekten takas (swap) alanına alındığında yada tekrar belleğe yüklendiğinde program sayıcıların ve tüm program kayıtlarının da bu sürece dahil edilmesi gerekir.

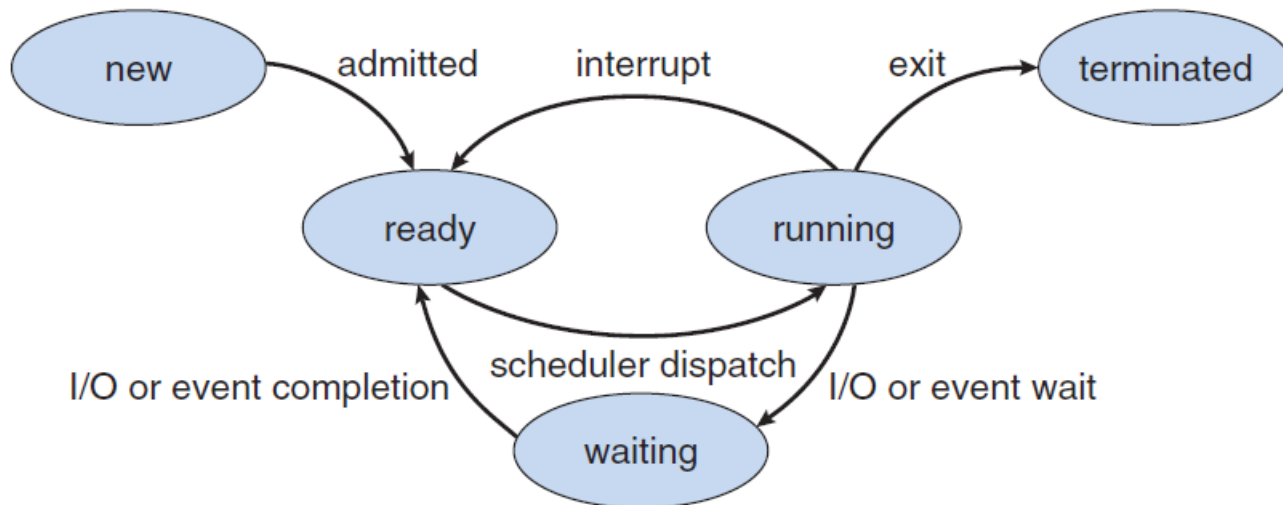


Süreçler / Processes



Süreç durumları

- Bir süreç yürütülürken farklı durumlara geçiş yapılır. Bu durumlar sürecin mevcut aktivite durumunu ifade etmek için kullanılır.
 - **New:** Süreç oluşturuldu
 - **Running:** Komutlar yürütülmeye başlandı.
 - **Waiting:** Süreç bazı olayların (klavye girişi, zamanlayıcı, işlemler arası mesaj vb.) ortaya çıkmasını bekliyor,
 - **Ready:** Sürecin çalışması için tüm kaynaklar hazır ancak CPU o anda bu süreci yürütmüyor,
 - **Terminated:** Süreç tamamlandı.



Süreç denetim bloğu (PCB)

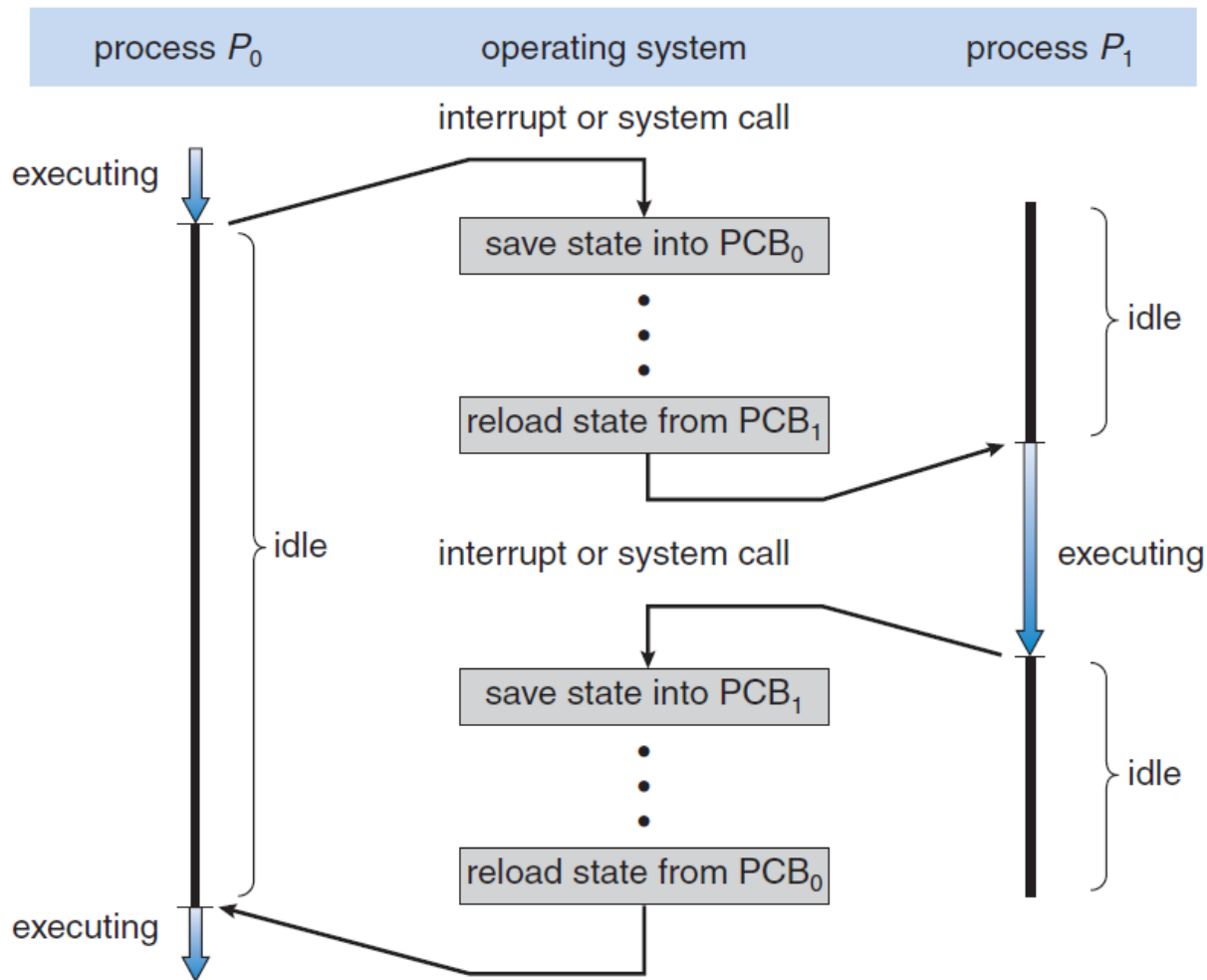
Her süreç, sürece özgü bilgiler içeren ve işletim sisteminde temsil edilen bir süreç denetim bloğuna (process control block - **PCB**) sahiptir.

- **Process state:** Süreç durumunu tanımlar (new, running vb.),
- **Program counter:** Bir sonraki işletilecek komutun adresi tutulur,
- **CPU registers:** CPU kaydedicilerinin (AX, PSW vb.) durumu tutulur.
- **CPU-scheduling information:** Sürecin öncelik, CPU planlaması vb planlamayla ilgili bilgiler tutulur,
- **Memory-management information:** İşletim sisteminin bellek yönetim sistemine bağlı olarak, sayfa/segment tabloları ve kaydedici limitleri gibi temel verileri tutar,
- **Accounting information:** CPU'nun gerçek kullanım zamanı, zaman limitleri, iş ve süreç numaraları gibi bilgileri tutar,
- **I/O status information:** Süreçte kullanılan G/Ç aygıtları ve açık dosyaları gibi bilgileri tutar.

process state
process number
program counter
registers
memory limits
list of open files
...

Süreç denetim bloğu (PCB)

CPU süreçler arası geçiş diagramı:



Süreçler arası geçiş

- Kesmeler, işletim sisteminin CPU’da yürütülen mevcut görevi bırakıp, başka bir sistem rutinine geçmesini sağlar.
- Bir kesme oluştuğunda, işletim sistemi, yürütülen görevi askıya alır ve daha sonra ilgili görevi yürütebilmek için süreç içeriğini saklar.
- Kesmelere benzer şekilde, CPU’da yürütülen bir sürecin, o süreç için ayrılan zaman kesiti (**time slice expired**) dolduğunda da benzer şekilde süreç geçişi gerçekleştirilir.
- Süreç içeriği, süreç durumu ve CPU kaydedicilerini içeren PCB (süreç kontrol bloğu) ile temsil edilir.
- Süreçler arası geçişte, CPU başka bir işlem yapmaz. Harcanan zaman donanıma ve süreç içeriğine bağlı olarak bir kaç ms civarında olabilir. Bu işlem çok sık tekrarlandığı için sistem performansını ciddi oranda etkilemektedir. Verimlilik için farklı yaklaşımlar ileriki konularda ele alınacaktır.

Süreçler arası geçiş - Mobil OS

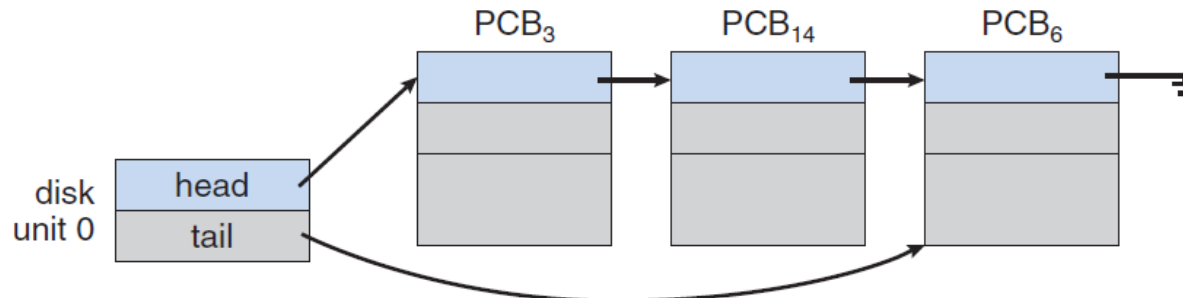
- **iOS** işletim sistemi 4 versiyonuna kadar çoklu görev (multitasking) yeteneğine sahip değildi.
- iOS 4'ten sonra arkaplan uygulamalarına izin verilmektedir. Fakat uzun süreli arkaplan uygulamalarına sonsuz çalışma izni verilmemektedir. Arkaplanda sürdürülmesine izin verilen görevler, tek ve sınırlı uzunluktaki işler (bir dosyanın indirilmesi gibi), push notification servisi ve uzun ömürlü özel arkaplan uygulamalarıdır (audio player gibi).
- iOS'un bu yaklaşımı verimli güç ve donanım kullanımı içindir.
- **Android** ise arkaplan uygulamaları için herhangi bir kısıt getirmemektedir.
- Uzun süre arkaplanda çalışması gereken uygulamaların hizmet vermeye devam edebilmesi için ilgili uygulamanın arkaplan servisi olarak yapılandırılması gerekir. Böylece arkaplanda uygulama askıya alınsa dahi çalışmaya devam edecektir.

Süreç planlama

Süreçler sisteme yüklenirken, sistemdeki tüm süreçlerin yer aldığı bir iş kuyruğuna (**job queue**) yerleştirilirler.

Ana bellekte yürütülmek için bekletilen ve durumu hazır - bekleme olan süreçler hazır kuyruk (**ready queue**) listesine alınırlar. Bu kuyruk linked list olarak kullanılır. Hazır kuyruksa bekletilen her PCB bir sonraki PCB'yi gösterir.

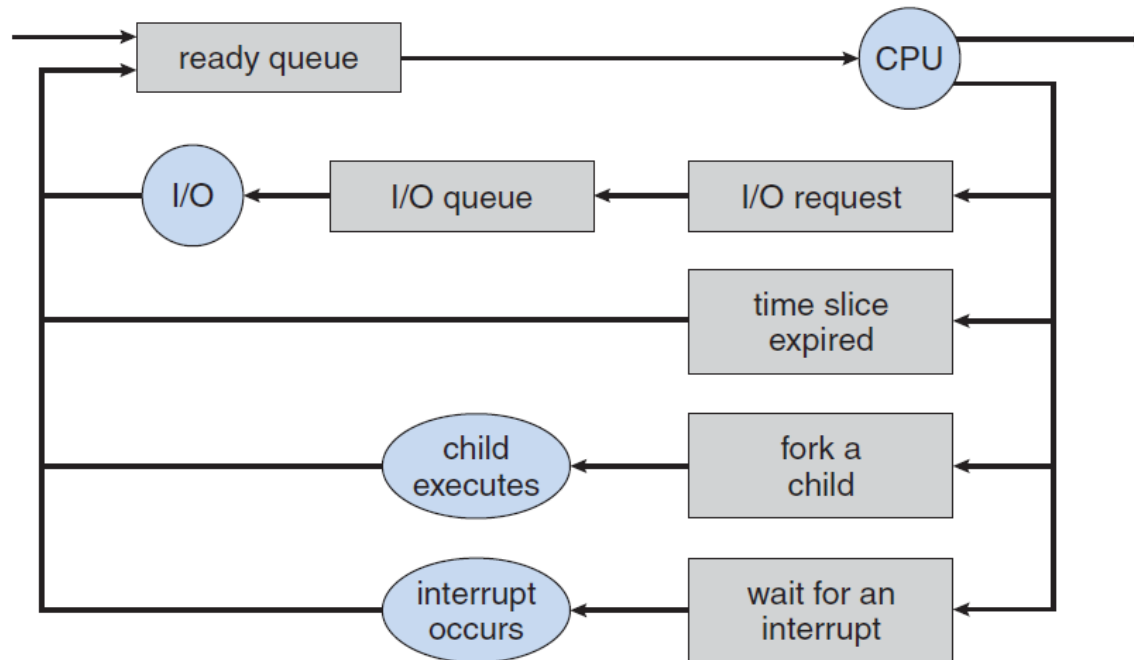
Bir süreç CPU tahsis edildiğinde, bu süreç bitinceye, kesilinceye yada beklemeye geçilinceye kadar çalıştırılır. Süreç bir G/Ç aygıtına erişmek istediğinde bekleme durumuna geçer. Disk gibi bir G/Ç aygıtından istekte bulunan pek çok süreç bu aygıtın meşgul olmasına neden olabilir. Bu nedenle G/Ç aygıtları için aygıt kuyruğu olarak ifade edilen (**device queue**) tutulur.



Süreç planlama

Yürütülecek süreç öncelikle **ready queue**'ye alınır. Süreç işlenirken CPU zamanlamasına uygun yürütülüp tekrar kuyruğa bırakılabilir yada bitirilebilir. Bazen süreç yürütülürken aşağıdaki durumlar oluşabilir;

- Süreç bir I/O isteğinde bulunabilir ve I/O kuyruğuna alınır ve bekler
- Süreç yeni bir alt süreç oluşturabilir ve child sürecin bitmesini bekler
- Bir kesme sonucunda CPU süreci bırakır ve süreç tekrar hazır süreç kuyruğuna taşınır.



Süreç planlama

Bir süreç, çalışma süresi boyunca sıkça kuyruklar arasında gezinir. İşletim sistemi kuyruklardaki süreçleri planlamak için farklı modlar kullanabilir.

Long-term scheduler: Toplu iş sistemlerinde yada ağır yük altında çalışan sistemlerde süreçlerin seyrek çalıştırılması için yapılan planlamadır. Bu sistemler genellikle süreç kuyruklarını sabit diskler üzerinden yürütürler.

Short-term scheduler: CPU, hazır süreç kuyruğundaki süreçler arasında çok hızlı (100 ms civarında) bir şekilde geçiş yaparak yürütür. Süreçlerin planlanması 10 ms sürer ve bu durumda CPU %9 boşta bekler. $10/(100+10)$.

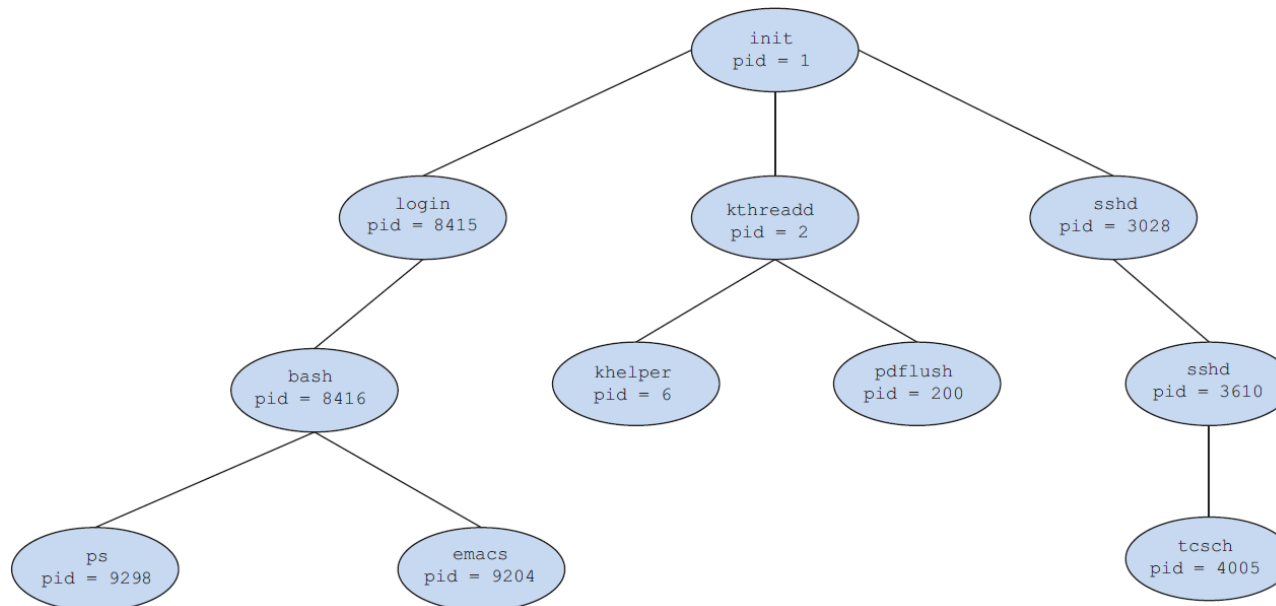
Medium-term scheduler: Sistem yükü fazla olduğunda bir veya daha fazla süreç birkaç saniye için hazır kuyruk dışına swaplanır ve küçük işlerin hızlıca tamamlanması ve sistemin temizlenmesi sağlanır. Unix – Windows

Tüm süreçler I/O yada CPU'ya bağlı olduğundan iyi bir zamanlamada I/O ve CPU birlikte dikkate alınmalıdır. Sadece I/O dikkate alınırsa hazır kuyruk boşta, CPU dikkate alınırsa da device kuyruğu boşta kalacaktır.

Süreç işlemleri - Süreç oluşturma

Çoğu işletim sisteminde süreçler dinamik olarak aynı zamanda çalıştırılabilir, oluşturulabilir ve silinebilir.

Bir süreç yürütülmesi esnasında yeni süreçler oluşturabilir. Sürecin kendisine ana (parent process) süreç, oluşturduklarına ise çocuk (child) süreç denir. Oluşturulan her süreç için sistemde onu tanımlayan bir **PID** (process identifier) değeri atanır. Şekilde linux sisteminde oluşturulan örnek bir süreç ağacı görülmektedir (ps -el).



Süreç işlemleri - Süreç oluşturma

Bir süreç child süreçler oluşturduğunda; (1) Parent süreç, child süreçlerle birlikte koşturaya devam eder, (2) Parent süreç child süreçlerin bir kısmı yada tamamı sonlanıncaya kadar bekleyebilir.

Yeni süreç için iki tür bellek alanı söz konusu olabilir; (1) Child süreç, parent süreçle aynı kod ve veri belleğini kullanan bir kopya olabilir, (2) Child süreç içerisinde yeni bir program yüklenebilir.

Unix tabanlı sistemlerde yeni süreç **fork()** sistem çağrısı ile oluşturulur. Yeni süreç, parent sürecin adres alanının kopyası ile oluşturulur. Yeni süreç yeni bir PID değerine sahiptir. İki süreçte yürütülmeye devam eder. Child sürecin geri dönüş değeri **sıfır** ise child sürecin yürütüldüğü, sıfırdan büyük ise parent sürecin child süreci beklemesi gerektiğini gösterir.

Genellikle **fork()** sistem çağrısından sonra parent sürecin bellek adres uzayını değiştirmek için **exec()** sistem çağrısı ile yeni bir süreç başlatımı sağlanır.

Süreç işlemleri - Süreç oluşturma/Posix

Aşağıda linux tabanlı parent/child süreç oluşturma örneği sunulmuştur;

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    char *arg_list[] = { "ls", "-l", "-a", NULL };
    pid = fork(); /* child süreç oluşturuldu */
    if (pid < 0) { /* hata oluştu */
        fprintf(stderr, "Hata\n");
        return 1;
    } else if (pid == 0) { /* child süreç işliyor */
        printf("Child süreç çalışıyor... child pid = %i\n", getpid());
        execvp("ls", arg_list);
    } else { /* parent süreç işliyor */
        /* child süreç tamamlanıncaya kadar parent süreç bekleyecek */
        printf("parent süreç çalışıyor, child'i bekle...");
        printf("parent pid = %i - child pid = %d\n\n", getpid(), pid);
        wait(NULL);
        printf("\nchild süreç tamamlandı\n");
    }
    return 0;
}
```

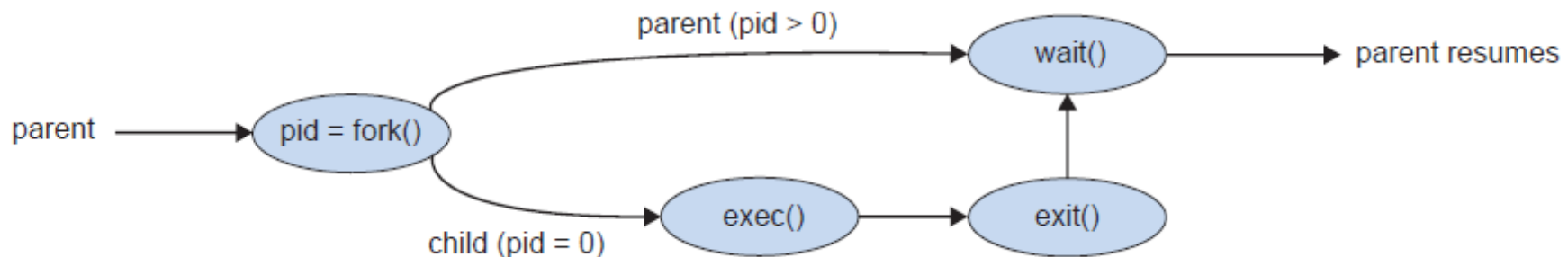
```
$ gcc -c surec.c
$ gcc surec.o -o surec
$ ./surec
```


Süreç işlemleri - Süreç oluşturma

Bu örnekte aynı program kopyasında çalışan iki farklı süreç görülmektedir. Burada dikkat edilmesi gereken durum child süreçteyken pid sıfır, parent süreçteyken pid sıfırdan büyük (gerçekte child sürecin pid değeridir) pozitif bir tam sayıdır.

Örnekte parent süreç çalıştırıldıktan sonra bir child süreç oluşturulmakta. Child süreç başka bir program yürütümünü tamamlayıcaya kadar parent süreç beklemektedir.

Bir child süreç oluşturulduğunda aynı hafıza adres alanı kullanıldığı için geri dönüş sürecinin iki kez (biri parent süreç, diğer child süreç) gerçekleştiği unutulmamalıdır.



Süreç işlemleri - Süreç oluşturma/Windows

```
#include <stdio.h>
#include <windows.h>
#include <locale.h>
int main(VOID) {
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    setlocale(LC_ALL, "Turkish");
    ZeroMemory(&si, sizeof(si)); /* bellek ayrılıyor */
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
    /* child süreç oluşturuluyor */
    if (!CreateProcess(NULL, /* komut istemi kullan */ "C:\\WINDOWS\\system32\\mspaint.exe",
        NULL, /* süreçten miras alma */ NULL, /* threadten miras alma */
        FALSE, /* miras almayı kapat */ 0, /* bayrak oluşturma */
        NULL, /* parent sürecin bloğunu kullan */ NULL, /* parent sürecin dizinini kullan */
        &si, &pi))
    {
        fprintf(stderr, "Hata, süreç oluşturulamadı");
        return -1;
    }
    /* parent süreç child süreci bekleyecek */
    printf("Parent süreç bekletiliyor...\n");
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child süreç tamamlandı");
    /* handle'lar kapatılıyor */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```

Süreç işlemleri - Süreç sonlandırma

Bir süreç **exit()** sistem çağrısını kullanarak işletim sistemine sonlanacağını bildirir. Bu noktada süreç, parent sürecine bir durum değeri (tam sayı) dönebilir. Böylece sürece tahsis edilen, fiziksel ve sanal bellek, açık dosyalar, G/Ç tamponları gibi tüm kaynaklar serbest bırakılır.

Bir süreç sonlanırken başka süreçleri de sonlandırabilir. Bu parent sürecin child süreçleri sonlandırması şeklinde gerçekleşir. Ancak bir sürecin sonlandırılabilmesi için sürecin PID'i, parent süreç tarafından biliniyor olması ile mümkündür. Parent süreç, (1) child süreç kaynak kullanım seviyesini aştığında, (2) görevi sona erdiğinde, (3) parent süreç sonlandırıldığında, gibi durumlarda child süreci bitirir.

Bazı sistemlerde child süreçlerin sonlandırılması işletim sistemi tarafından gerçekleştirilir.

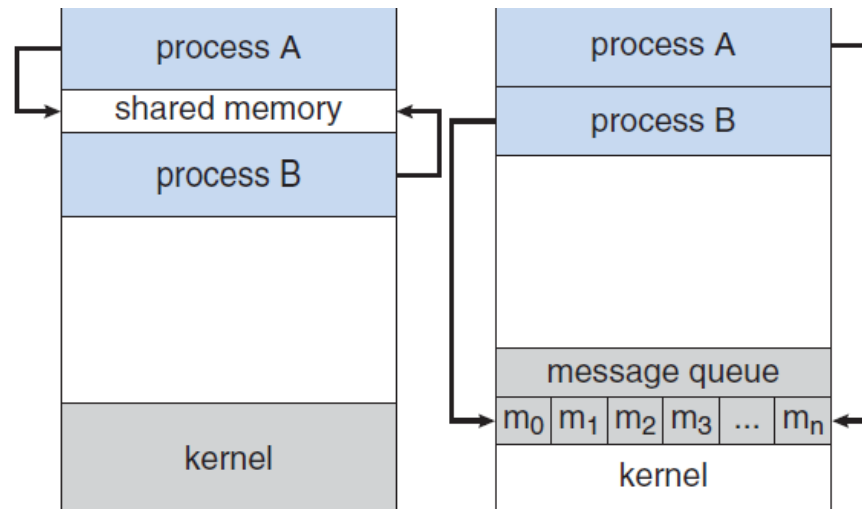
Bir child süreç sonlandığında, parent süreç, **wait()** sistem çağrısıyla süreç durum kodunu süreç çıkış kodu olarak işleyinceye kadar, kullanılan kaynaklar serbest bırakılamaz ve kısa bir süre **zombi süreç** olarak kalır.

```
exit(1); /* child çıkış kodu */
```

```
pid = wait(&status); /*parent'e dönüş*/
```

Süreçler arası iletişim

- İşletim sistemlerinde aynı anda yürütülen süreçler birbirlerinden bağımsız olabildikleri gibi işbirliği içerisinde de olabilirler. Bir süreç başka bir süreçle veri paylaşarak işbirliği yapabilir.
- Süreçler arasında işbirliği; bilgi paylaşımı, hesaplama hızı, modülerlik ve kullanım kolaylığı gibi farklı sebeplerle olabilir.
- Süreçler arası iletişim **InterProcess Communication (IPC)** olarak bilinir.
- Temelde (1)**shared-memory** ve (2)**message-passing** olarak bilinen iki yöntem vardır.
- Birinci modelde, süreçler arasında bir bellek alanı paylaşılmak üzere kurulmuştur. Süreçler bu bellek alanı üzerinden verileri okuyup yazabilirler. İkinci modelde süreçler birbirlerine mesaj geçişi yaparak veri alışverişi yapabilirler.



Süreçler arası iletişim - shared-memory

- **Shared-memory**, süreçler arasında veri alışverişi için paylaşılan bir bellek alanının kurulmasını gerektirir. Bu yöntemde birinci süreç kendi adres uzayından bir paylaşılan bellek segmenti oluşturur ve diğer süreç bu segmenti kendi adres alanına ekler.
- Normalde işletim sistemleri iki süreç arasında birbirlerinin bellek alanlarına erişimine izin vermez. Ancak bu yöntem kullanıldığında paylaşılan bellek segmenti işletim sisteminin kontrolünden çıkar ve sorumluluk süreçlere aittir.
- Bu yaklaşımda bellek alanı bir tampon olarak konumlandırılır. Tampon alanını kullanan süreçleri alıcı-verici olarak düşünecek olursak; sınırsız tampon (**unbounded buffer**) yada sınırlı tampon (**bounded buffer**) olabilir. Tampon türüne göre alıcı ve verici tampon durumunu boş/dolu kontrol etmelidir.

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- in* değişkeni tamponun ilk boş konumunu,
out değişkeni tamponun ilk dolu konumunu,
 Eğer $in == out$ ise tampon boş,
 Eğer $((in+1) \% BUFFER_SIZE == out)$ ise dolu.
- Ancak sadece $BUFFER_SIZE-1$ kadar alan kullanılabilir

Süreçler arası iletişim - shared-memory

- Shared memory yönteminde alıcı-verici arasında tamponun boş-dolu olması durumunda karşılaşılan problem, üretici/tüketici problemi şeklinde ifade edilir. Bu problem aşağıdaki kontrollerle engellenebilir.

ARAŞTIRMA: Bu yöntemi kullanan iki süreçten oluşan bir chat uygulaması...

```
item send_data;
while (true) {
    while (((in + 1) % BUFFER_SIZE) == out); /*tampon dolu*/
    /* paylaşılan belleğe gönder */
    buffer[in] = send_data;
    in = (in + 1) % BUFFER_SIZE;
}
```

```
item recv_data;
while (true) {
    while (in == out); /* tampon boş */
    /*paylaşılan bellekten oku*/
    recv_data = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
}
```

IPC Örnekleri – shared memory (posix)

/*yaz.c*/

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <unistd.h>
int main()
{
    const int SIZE = 4096; /*Boyut*/
    const char *name = "SIVAS"; /*Nesne Adı*/
    const char *message_0 = "Merhaba "; /*1. Mesaj*/
    const char *message_1 = "Dünya!\n"; /*2. Mesaj*/
    int shm_fd; /*shared memory dosya tanımlayıcı*/
    void *ptr; /*shared memory nesne göstericisi*/

    /*shm nesnesi oluşturuluyor*/
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, SIZE); /*Nesne boyutu ayalanıyor*/
    /* Nesne, memory mapped file içerisine kuruluyor*/
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);
    sprintf(ptr, "%s", message_0); /*belleğe yazılıyor*/
    ptr += strlen(message_0);
    sprintf(ptr, "%s", message_1);
    ptr += strlen(message_1);
    return 0;
}
```

/*oku.c*/

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
int main()
{
    const int SIZE = 4096;
    const char *name = "SIVAS";
    int shm_fd;
    void *ptr;

    shm_fd = shm_open(name, O_RDONLY, 0666);
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);
    printf("%s", (char *)ptr); /*Oku ve ekrana yaz*/
    shm_unlink(name); /*bağlantıyı kes*/
    return 0;
}
```

#Makefile

```
OPT_GCC = -std=c99 -Wall -Wextra
OPT = -D_XOPEN_SOURCE=700
```

```
all: oku yaz
```

```
oku: oku.c
```

```
gcc $(OPT_GCC) $(OPT) -o oku oku.c -lrt
```

```
yaz: yaz.c
```

```
gcc $(OPT_GCC) $(OPT) -o yaz yaz.c -lrt
```

```
run: oku yaz
```

```
./yaz
```

```
./oku
```

```
clean:
```

```
rm -f oku yaz
```

Süreçler arası iletişim - message-passing

- **Message-passing**, süreçlerin, aynı bellek alanını paylaşmadan, verilerini iletebilmek için kullanılır. Shared-memory yönteminde süreçler paylaştıkları bellek segmentini bilmek ve açıkça program içerisinde tanımlamak zorundadır. Bu nedenle message-passing bir ağ üzerinde iletişim halinde yada dağıtık mimarideki uygulamalar için (chat, file transfer vb.) uygun bir yöntemdir.
- Message-passing yöntemi `send(message)` ve `receive(message)` gibi iki temel operasyona sahiptir.
- İletişim kurmak isteyen P ve Q gibi iki süreç arasında mantıksal bir bağlantı kurulmalıdır. Bu mantıksal bağlantı farklı yöntemlerle kullanılabilir;

1. Direct/Indirect: Doğrudan bağlantıda iki süreç iletilerinde doğrudan hedefin adını (alıcı/verici) belirtmelidir.

`send(P, message) / receive(Q, message)`

Indirect iletişimde ise alıcı ve gönderici merkezi bir server (mailbox gibi) üzerinden iletişim kurabilirler. (A burda mail sunucusu)

`send(A, message) / receive(A, message)`

Süreçler arası iletişim - message-passing

2. Senkron/Asenkron: Senkron yada asenkron iletişimde alıcı verici send()/receive() süreçlerinin senkronizasyonunda farklı yöntemler kullanılabilir. Bunlar **(1)Blocked send**, **(2)Nonblocking send**, **(3)Blocking receive** **(4)Nonblocking receive**'dir.

- 1. ve 3. yöntemler senkron süreç iletişimde; gönderici süreç, iletilen mesaj alıcı süreç tarafından alınana kadar, alıcı süreç de mesaj varolup alınıncaya kadar bloklanır.
- 2. ve 4. yöntemler asenkron süreç iletişimde; süreçler herhangi bir durumu beklemeksizin işlemlerine devam edebilirler.

Senkron iletişim, üretici/tüketici problemi için uygun çözüm sunmaktadır.

```
item send_data;  
while (true) {  
    send(send_data); //gönderim gerçekleşene kadar bekle.  
}
```

```
item recv_data;  
while (true) {  
    receive(recv_data); //tamponda veri olana kadar bekle  
}
```

Süreçler arası iletişim - message-passing

3. Sınırlı/Sınırsız tampon: Alıcı/verici arasında paylaşılan mesajlar işleninceye kadar geçici kuyruklarda bekletilirler. Bu tamponlama süreçlerinde farklı yöntemler kullanılabilir;

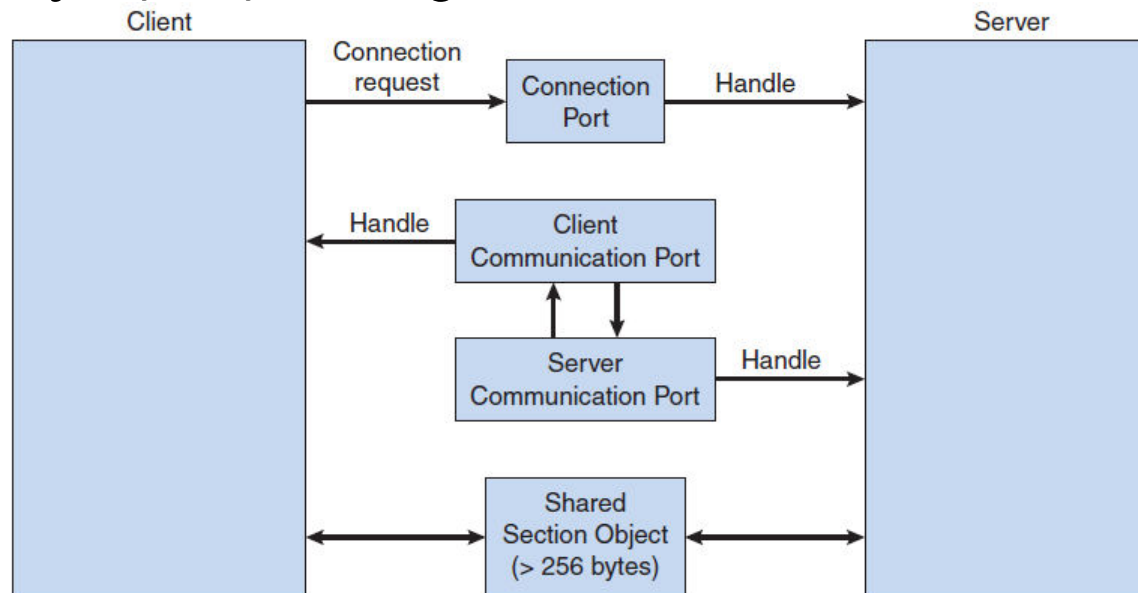
- **Sıfır Tampon:** Kuyruk yoktur. Alıcı mesajı işlemeden yeni mesaj iletilemez.
- **Sınırlı Tampon:** n tane uzunluğunda tampon doluncaya kadar mesaj iletilebilir. Tampon dolunca iletim engellenmelidir. Tampon boşsa alıcı taraf bekletilmelidir.
- **Sınırsız Tampon:** Tamponda sınır yoktur, gönderici teorikte sonsuz gönderim yapabilir. Ancak alıcı tamponun boş olup olmadığını kontrol etmelidir.

Sıfır tampon yöntemi **tamponsuz (no-buffering)** yöntem olarak tanımlanırken, diğer iki yöntem **otomatik tamponlama (automatic buffering)** yöntemi olarak da bilinir.

IPC Örnekleri - message passing (windows)

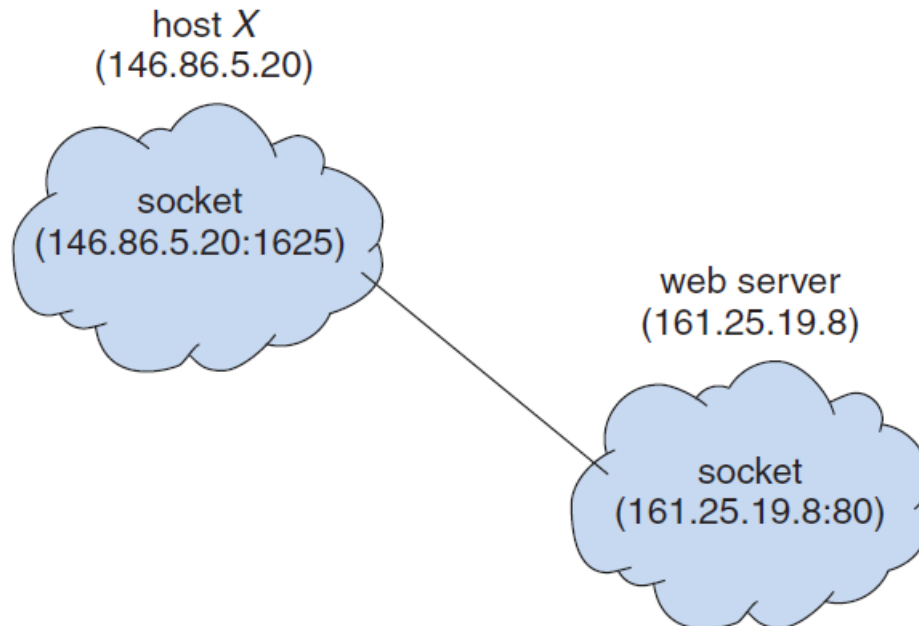
Windows işletim sisteminde message-passing yöntemi ileri yerel yordam çağrısı (advanced local procedure call - ALPC) olarak tanımlanır. Yerel iki süreç arasında bağlantı kurmak ve sürdürmek için **port nesnesi** kullanılır. Bu port nesneleri **connection port** ve **communication port** olmak üzere iki türdür.

- **Connection port**, sunucu üzerinde tüm süreçler tarafından erişilebilen ve istemcilerin bağlantı isteklerini karşılayan nesnedir.
- **Communication port**, sunucuya iletilen bağlantı isteği başarıyla kurulduğunda istemci-sunucu arasında kurulan özel kanaldır. Bu kanal, istemci-sunucu çifti arasında mesaj alışverişinde ve geribildirimlerde kullanılır.



İstemci/Sunucu Sistemleri - Soketler

- Soketler iki düğüm arasında, taşıma katmanınca tanımlanan uç noktalardır,
- Bir ağ üzerinden kurulacak iletişim için her süreç bir soket çifti tanımlar,
- Soket bir IP adresi ve bir port numarası çiftinden oluşur, (161.25.19.8:1625)
- Port numaraları 2 byte'tır ve 1024'ün altındaki portlar genellikle yaygın/bilinen uygulama protokolleri tarafından kullanılır. (ftp-21, http-80, https-443 vb.)
- Bir istemci süreci bir bağlantı isteği oluşturduğunda bu sürece 1024'ün üzerinde boş bir port numarası atanır. Şekilde bir web server'a bağlantı kurmak isteyen istemci ile sunucu arasındaki soket tanımı görülmektedir.



İstemci/Sunucu Sistemleri - Soketler

- Programlama dilleri soket oluşturmak ve kullanmak için API'lar sunarlar.
- Java programlama dili, TCP, UDP ve yayın için MulticastSocket tanımlamaları yapmak için uygun sınıflar sunar. Örnek istemci/sunucu soket uygulaması;

```
import java.net.*;
import java.io.*;
public class server {
    public static void main(String[] args) {
        try {
            ServerSocket sock = new ServerSocket(6013);
            while (true) {
                Socket client = sock.accept();
                PrintWriter pout = new
                    PrintWriter(client.getOutputStream(),
                        true);
                pout.println("Sunucu Mesajı");
                client.close();
            }
        } catch (IOException ioe) {
            System.err.println(ioe);
        }
    }
}
```

```
import java.net.*;
import java.io.*;
public class client {
    public static void main(String[] args) {
        try {
            Socket sock = new Socket("127.0.0.1", 6013);
            InputStream in = sock.getInputStream();
            BufferedReader bin = new BufferedReader(new
                InputStreamReader(in));
            String line;
            while ((line = bin.readLine()) != null)
                System.out.println(line);
            sock.close();
        } catch (IOException ioe) {
            System.err.println(ioe);
        }
    }
}
```

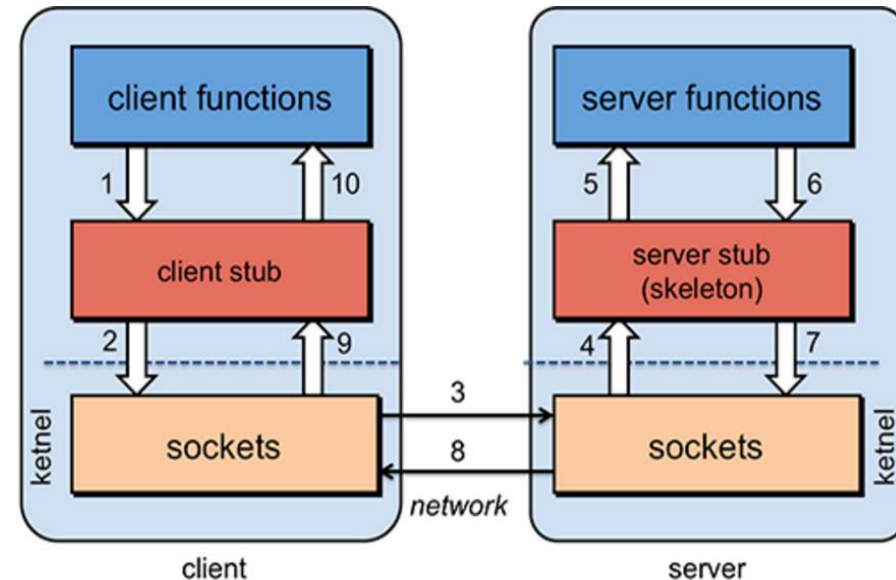
- Burada TCP soket açan server uygulaması 6013 portunu dinlemekte ve accept() metodu ile istemci/sunucu soket çifti kurulmaktadır.

İstemci/Sunucu - Uzak Prosedür Çağırımı/RPC

- Uzak makinelerdeki süreçler arasında mesaj geçişi tabanlı veri iletişimi için yerel prosedür çağırımına benzer, uzak prosedür çağırımı geliştirilmiştir.
- Bir sistem, uzak prosedür çağırımı taleplerine cevap verebilmek için bu servisin hizmet verdiği bir **port**'u yapılandırır.
- Bağlantının (istemci-sunucu) iki ucunda da **stubs** (uzak çağırımlar arasında parametre dönüştürücüler – soap mimarisi gibi) bulunur.
- Uçlar arasında stub'lar manual olarak yazılabileceği gibi **Interface Definition Language – IDL** (bkz. Microsoft midl.exe) kullanılarak da geliştirilebilir.
- Tanımlanan stub'lar sunucu üzerindeki portu ve parametreleri bulur.
- Parametreler ağ üzerinde iletilebilir formda serileştirilir (**marshalling**).
- İstemci tarafındaki stub, sunucu portunu bulur ve parametreleri serileştirerek (marshalling) sunucu'ya geçer. Sunucu stub, ilgili parametreleri prosedüre geçer ve sonucu benzer işlemlerle istemciye döner.
- Geçilen parametrelerde uzak sistemler arasında uyumluluk sorunu olabilir (big-endian, little-endian → x86 little-endian, JVM big-endian kullanır). Bu sorunu çözmek için **external data representation (XDR)** kullanılır.
- Uzak sistemlerdeki tüm prosedürlere özgü bir port ayrılabilceği gibi bir randevu (**matchmaker**) sunucusu üzerinden ilgili uzak prosedürlere yönlendirme yapılabilir.

İstemci/Sunucu - Uzak Prosedür Çağırımı/RPC

1. İstemci fonksiyonu, istemci stub'ı çağırır.
2. Stub isteği sunucu soketine iletir
3. İstek (TCP/UDP) sunucu soket aktarılır
4. Sunucu stub'ı mesajı alır, parametreleri unmarshalling eder.
5. Sunucu stub'ı parametrelerle birlikte server fonksiyonu çağırır.
6. Sunucu fonksiyonu tamamlandıktan sonra geri dönüş değerlerini sunucu stub'a aktarır.
7. Sunucu stub'ı aldığı değerleri serilize eder ve ağ paketini oluştur.
8. Mesaj istemci soketine iletilir.
9. İstemci stub mesajı okur.
10. İstemci stub geri dönüş değerini istemci fonksiyonuna geçer.



İstemci/Sunucu - RPC - rpcgen örneği

```
# rpcinfo
# apt-get install rpcbind
```

```
/*testprc.idl*/
program RPCPROG {
    version RPCVERS {
        string serverproc(void) = 1;
    } = 1;
} = 0x12345678;
```

```
# rpcgen -C testprc.idl
# rpcgen -a -C testprc.idl
```

```
/*testrpc_client.c*/
....
result_1 = serverproc_1((void*)&serverproc_1_arg, clnt);
if (result_1 == (char **) NULL) {
    clnt_perror (clnt, "call failed");
}else{
    printf("Dönen mesaj=%s\n",*result_1);
}
...
```

```
# make -f Makefile.testrpc
# ./testrpc_server | ./testrpc_client localhost
```

```
/*testrpc_server.c*/
serverproc_1_svc(void *argp, struct svc_req *rqstp)
{
    static char * result;
    static char msg[256];

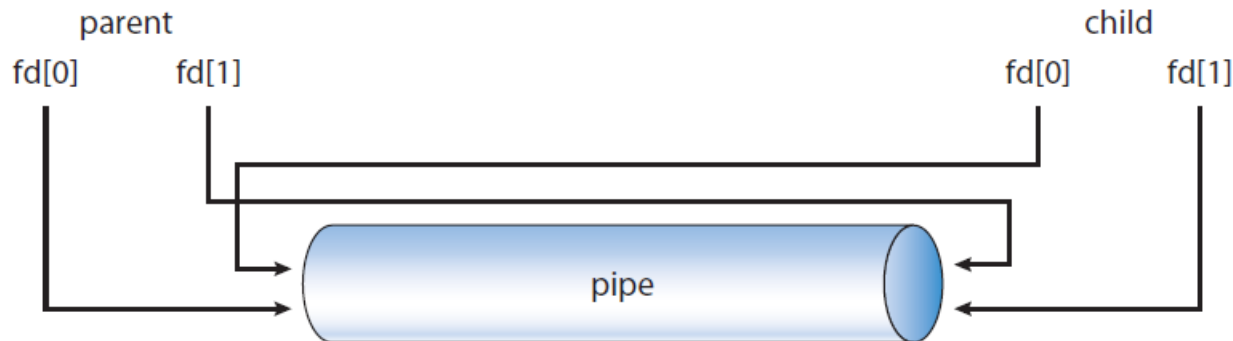
    printf("prosedür çağrıldı...\n");
    strcpy(msg, "Merhaba dünya");
    result = msg;
    return &result;
}
```


İstemci/Sunucu - Kanal (pipe)

- Pipe iki süreç arasında iletişim kurmak için bir kanal görevi yapar,
- UNIX sistemlerinde IPC mekanizması olarak kullanılmaya başlanmıştır.
- Kanal kullanımında dikkat edilesi gereken hususlar;
 1. Kanalın tek yönlü mü çift yönlümü olduğu,
 2. İletişim çift yönlü ise, half-duplex'de aynı anda sadece bir yönde iletim, full-duplex ise aynı anda çift yönlü iletişim sağlanabilir,
 3. İletişim kuracak süreçler arasında ilişki olmalı (parent-child),
 4. Kanallar aynı makine üzerinde yada ağ üzerinde kurulabilir.
- Kanallar windows ve unix'te sıralı kanal (Ordinary Pipes) yada adlandırılmış kanal (named pipes) olarak tanımlanabilir.

İstemci/Sunucu - Kanal / Ordinary Pipes

- Sıralı kanallar üretici/tüketici (producer–consumer) mantığına göre iletişim kurarlar.
- Üretici (producer) kanala yazarken (write-end), tüketici (consumer) bu veriyi kanaldan okur (read-end),
- Sıralı kanallar tek yönlüdür, çift yönlü iletim için iki kanal tanımlanmalıdır.
- Unix sistemlerde kanal, `pipe(int fd[])` ile tanımlanır.
- Burada `fd(0)` kanaldan dosya okuma, `fd(1)` yazma amaçlıdır.
- Sıralı kanallara süreç dışından erişilemez, dolayısıyla `fork()` ile child süreçler ile iletişim kurulabilir.



İstemci/Sunucu - Ordinary Pipes – posix örneği

```

/*pipe.c*/
#include <sys/types.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#define BUFFER_SIZE 25
#define READ_END 0
#define WRITE_END 1
int main(void) {
    char write_msg[BUFFER_SIZE]="test";
    char read_msg[BUFFER_SIZE];
    int fd[2];
    pid_t pid;
    if(pipe(fd)==-1){/*Kanal oluştur*/
        fprintf(stderr,"Kanal Hatası");
        return 1;
    }
    pid = fork(); /*child süreç tnm*/
    if (pid < 0) {
        fprintf(stderr, "Fork hatası");
        return 1;
    }

```

```

    if (pid > 0) { /* parent süreç */
        close(fd[READ_END]); /*kanalı kapa*/
        write(fd[WRITE_END], write_msg,
            strlen(write_msg)+1);
        close(fd[WRITE_END]);
        wait(NULL);
        printf("\nchild süreç tamamlandı\n");
    } else { /* child süreç */
        close(fd[WRITE_END]);
        read(fd[READ_END], read_msg, BUFFER_SIZE);
        printf("read %s\n",read_msg);
        close(fd[READ_END]);
    }
    return 0;
}

```

```

$ gcc -c pipe.c
$ gcc pipe.o -o pipe
$ ./pipe

```

İstemci/Sunucu - Ordinary Pipes – win örneği

```
#include <stdio.h> /*parent.c*/
#include <stdlib.h>
#include <windows.h>
#define BUFFER_SIZE 25
int main(VOID){
    HANDLE ReadHandle, WriteHandle;
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    char message[BUFFER_SIZE] = "
    DWORD written;
    SECURITY_ATTRIBUTES sa = {siz
    ZeroMemory(&pi, sizeof(pi));
    if (!CreatePipe(&ReadHandle,
        fprintf(stderr, "Kanal Ha
        return 1;
    }
    GetStartupInfo(&si);
    si.hStdOutput = GetStdHandle(
    si.hStdInput = ReadHandle;
    si.dwFlags = STARTF_USESTDHAN
    SetHandleInformation(WriteHan
    CreateProcess(NULL, "child.ex
    CloseHandle(ReadHandle);
    if (!WriteFile(WriteHandle, m
        fprintf(stderr, "Kanala yazılamadı.");
    CloseHandle(WriteHandle);
    WaitForSingleObject(pi.hProcess, INFINITE);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    return 0;
}
```

```
#include <stdio.h> /*child.c*/
#include <windows.h>
#define BUFFER_SIZE 25
int main(VOID)
{
    HANDLE ReadHandle;
    CHAR buffer[BUFFER_SIZE];
    DWORD read;
    ReadHandle = GetStdHandle(STD_INPUT_HANDLE);
    if (ReadFile(ReadHandle, buffer, BUFFER_SIZE, &read, NULL))
        printf("Kanalardan okunan mesaj: %s\n",buffer);
    else
        fprintf(stderr, "Kanalardan okuma hatası");
    return 0;
}
```

İstemci/Sunucu - Kanal / Named Pipes

- Sıralı kanallarda kanal kullanımı süreçlerin yürütülmesine bağlıdır. Kanal süreç sona erdiğinde sonlandırılır.
- Adlandırılmış kanallarda iletişim çift yönlüdür ve süreçler arasında parent-child ilişkisi olma zorunluluğu yoktur.
- Bu kanallarda süreçler bittikten sonra da kanal var olmaya devam edecektir.
- UNIX sistemlerde bu tip kanallar yarı çift yönlü olarak konuşlandırılır. Ayrıca aynı makine üzerinde kullanılabilirler (open, close, write ve read metodu ile dosya üzerinden kullanılır). İlgili dosya disk üzerinden silininceye kadar kanal kullanılmaya devam eder.
- Windows sistemlerde ise bu kanallar tam çift yönlü iletişimi destekler ve uzak makineler üzerinden de bu kanallara erişilebilir.
- Windows sistemlerde CreateNamedPipe ile kanal tanımlanıp, ConnectNamedPipe ile kanala bağlanılır, ReadFile ve WriteFile ile kanaldan okunup yazılabilir.

İstemci/Sunucu - Named Pipes - Win. Örneği

```

/*server.c*/

#include <stdio.h>
#include <windows.h>

#define g_szPipeName "\\.\Pipe\MyNamedPipe"

#define BUFFER_SIZE 1024 //1k
#define ACK_MESG_RECV "server cevap mesaji"

int main(int argc, char* argv[])
{
    HANDLE hPipe;

    hPipe = CreateNamedPipe(
        g_szPipeName,          // pipe name
        PIPE_ACCESS_DUPLEX,    // read/write access
        PIPE_TYPE_MESSAGE |    // message type pipe
        PIPE_READMODE_MESSAGE | // message-read mode
        PIPE_WAIT,             // blocking mode
        PIPE_UNLIMITED_INSTANCES, // max. instances
        BUFFER_SIZE,           // output buffer size
        BUFFER_SIZE,           // input buffer size
        NMPWAIT_USE_DEFAULT_WAIT, // client time-out
        NULL);                 // default security attribute

    if (INVALID_HANDLE_VALUE == hPipe)
    {
        printf("\nKanal olusturulurken hata oldu: %d", GetLastError());
        return 1;
    }
    else
    {
        printf("\nCreateNamedPipe() basarili.");
    }

    printf("\nClient'in baglanmasi bekleniyor...");
    BOOL bClientConnected = ConnectNamedPipe(hPipe, NULL);
    if (FALSE == bClientConnected)
    {
        printf("\nClient baglanti hatasi: %d", GetLastError());
        CloseHandle(hPipe);
        return 1;
    }
    else
    {
        printf("\nConnectNamedPipe() basarili.");
    }
}

```

```

/*client.c*/

#include <stdio.h>
#include <windows.h>

//Name given to the pipe
#define g_szPipeName "\\.\Pipe\MyNamedPipe"
//Pipe name format - \servername\pipe\pipename
//This pipe is for server on the same computer,
//however, pipes can be used to
//connect to a remote server

#define BUFFER_SIZE 1024 //1k
#define ACK_MESG_RECV "Message received successfully"

int main(int argc, char* argv[])
{
    HANDLE hPipe;

    hPipe = CreateFile(
        g_szPipeName, // pipe name
        GENERIC_READ | // read and write access
        GENERIC_WRITE,
        0,             // no sharing
        NULL,           // default security attributes
        OPEN_EXISTING, // opens existing pipe
        0,             // default attributes
        NULL);         // no template file

    if (INVALID_HANDLE_VALUE == hPipe)
    {
        printf("\nServer'a baglanirken hata oldu: %d", GetLastError());
        return 1;
    }
    else
    {
        printf("\nCreateFile() basarili.");
    }

    char szBuffer[BUFFER_SIZE];

    printf("\nServer'a iletilecek mesaji gir: ");
    gets(szBuffer);

    DWORD cbBytes;

    //Send the message to server

```